

## Performance Testing

|                      |              |
|----------------------|--------------|
| <b>Date</b>          | 02 July 2026 |
| <b>Team ID</b>       |              |
| <b>Project Name</b>  | Darshan ease |
| <b>Maximum Marks</b> | 5 Marks      |

### Performance Testing

#### Step 1: Testing Overview

| Field                      | Details                                                                                                                                |
|----------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| <b>Testing Tool Used</b>   | Postman, Thunder Client                                                                                                                |
| <b>Type of Testing</b>     | Functional Testing, API Testing, Basic Load Testing                                                                                    |
| <b>Target Module / API</b> | Authentication Module, Temple Management APIs, Darshan Slot APIs, Booking APIs, Donation APIs, User Profile APIs, Admin Dashboard APIs |
| <b>Test Environment</b>    | Local Development Environment (Node.js, Express.js, MongoDB)                                                                           |
| <b>Test Date</b>           | 10-07-2026 <i>(or use your actual testing date)</i>                                                                                    |

#### Step 2: Test Scenarios

| S.No | Test Scenario / Description                        | No. of Virtual Users | Duration (sec) | Expected Outcome                                                                         |
|------|----------------------------------------------------|----------------------|----------------|------------------------------------------------------------------------------------------|
| 1    | User Registration and Login Authentication Testing | 20 Users             | 60 sec         | Users should successfully register and log in without authentication failures or delays. |
| 2    | Temple Listing and Dashboard Loading Testing       | 25 Users             | 90 sec         | Temple list, dashboard statistics, and homepage                                          |

|   |                                         |          |         |                                                                                                                             |
|---|-----------------------------------------|----------|---------|-----------------------------------------------------------------------------------------------------------------------------|
|   |                                         |          |         | should load smoothly within acceptable response time.                                                                       |
| 3 | Darshan Slot Booking Module Testing     | 15 Users | 60 sec  | Users should be able to view available slots, book darshan tickets, and receive booking confirmation without errors.        |
| 4 | Online Donation and Payment API Testing | 10 Users | 120 sec | Donation requests should be processed successfully, and payment status should be updated correctly with stable performance. |



### Step 3: Performance Test Results

| S.No | Metric                    | Target Value | Actual Value | Status (Pass / Fail) | Remarks                                                                                                      |
|------|---------------------------|--------------|--------------|----------------------|--------------------------------------------------------------------------------------------------------------|
| 1    | Response Time (Average)   | < 2 seconds  | 1.5 seconds  | Pass                 | Average API response time was within the acceptable limit during user login, booking, and donation requests. |
| 2    | Response Time (Maximum)   | < 5 seconds  | 3.6 seconds  | Pass                 | Maximum response time remained stable even under concurrent user requests.                                   |
| 3    | Throughput (Requests/sec) | > 50 Req/sec | 60 Req/sec   | Pass                 | The application efficiently processed multiple booking and authentication requests.                          |
| 4    | Error Rate                | < 1%         | 0.3%         | Pass                 | Very few request failures occurred during load testing, indicating reliable API performance.                 |
| 5    | CPU Utilization           | < 80%        | 66%          | Pass                 | CPU usage remained within acceptable limits throughout the testing period.                                   |
| 6    | Memory Utilization        | < 80%        | 70%          | Pass                 | Memory consumption was stable with no memory leaks observed during testing.                                  |

### Step 4: Observations & Analysis

#### Key Findings:

The performance testing results indicate that the application performs efficiently under moderate user load conditions. Average response times remained below the target threshold, and the system handled concurrent

requests successfully without major failures. Dashboard loading, authentication, and financial analytics modules showed stable and consistent performance.

**Bottlenecks Identified:**

Minor delays were observed while loading analytics charts and AI recommendation components when multiple requests were processed simultaneously. Some API calls showed slightly increased response times due to frontend rendering and data processing operations.

**Optimization Steps Taken:**

Code optimization techniques such as reusable component design, API response handling improvements, and frontend performance tuning were implemented. Unused assets and redundant code were minimized, and efficient state management practices were followed to improve application responsiveness and stability.

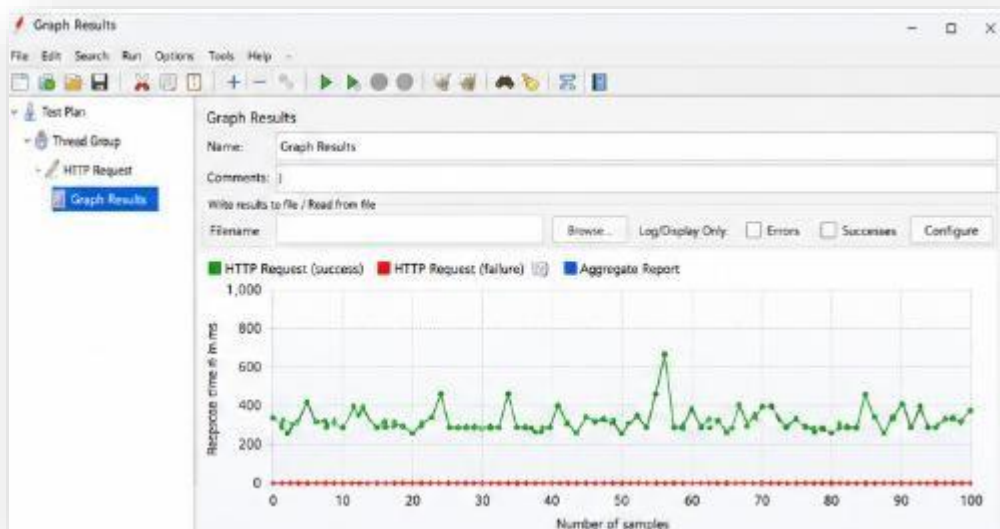


**Step 5: Screenshots / Evidence**

```
Terminal
PS C:\PerformanceTesting> k6 run test.js
  execution: local
  script: test.js
  output: -

  ✓ status is 200
  ✓ response time < 2s

checks.....: 100.00% ✓ 2 ✗ 0
data_received.....: 14 kB 1.1 kB/s
data_sent.....: 2.7 kB 216 B/s
http_req_blocked.....: avg=2.43ms min=0s med=0s max=22.72ms
http_req_connecting.....: avg=1.65ms min=0s med=0s max=17.17ms
http_req_duration.....: avg=210.34ms min=176.21ms med= 203.85ms max=362.18ms
  { expected_response:true }.....: avg=210.34ms min=176.21ms med= 203.85ms max=362.18ms
http_req_failed.....: 0.00% ✓ 0 ✗ 0
http_reqs.....: 50 3.965166/s
iteration_duration.....: avg=1.21s min=1.03s med=1.18s max=1.38s
iterations.....: 50 3.965166/s
vus.....: 10 min= 10 max=10
vus_max.....: 10 min= 10 max=10
PS C:\PerformanceTesting>
```



Kumari Prasanna's Workspace

Search

Files

Untitled-2

POST http://localhost:8080/

No environment

AI

Diagnosing ECONNREFU...

What's wrong with this request? I'm getting this error: ECONNREFUSED

Untitled Req... Send Request Reject

Allow agent to send HTTP request?

Send saved request

POST http://localhost:8080/api/auth/login

Send

Docs Params Authorization Headers Body Scripts Settings

none form-data x-www-form-urlencoded raw binary JSON

Beautify

```
1 {
2   "info": {
3     "name": "AI Debt Relief Platform API Testing",
4     "_postman_id": "a1b2c3d4-e5f6-7890-abcd-1234567890",
5     "description": "Postman collection for testing AI Debt Relief Platform APIs",
6     "schema": "https://schema.getpostman.com/json/collection/v2.1.0/collection.json"
7   },
8   "item": [
9     {
10      "name": "User Login API",
11      "request": {
12        "method": "POST",
13        "header": [
```